

SETTINGS AND PERMISSIONS

Choose a sandbox environment

Compare Claude Code sandbox options: the built-in sandboxed Bash tool, sandbox runtime, dev containers, Docker, and VMs. Choose the right isolation for your threat model.

Isolating Claude Code limits what a session can read, write, and reach on the network. This matters most when you let Claude work with fewer permission prompts, run it unattended, or point it at code you do not fully trust.

Claude Code can run in several kinds of isolated environments, ranging from a lightweight per-command sandbox to a fully separate virtual machine. This page compares them by what they isolate and what they require, helps you choose one for your threat model, and shows how to enforce that choice across an organization.

 For the broader security model, see [Security](#). For Agent SDK deployments, see [Secure deployment](#).

Compare sandboxing approaches

The first two approaches in the table below run on the host operating system without containers. The rest place Claude Code inside a container or virtual machine.

Approach	What is isolated	Requires Docker	Setup effort
<u>Sandboxed Bash tool</u>	Bash commands and their child processes	No	Minimal on macOS; low on Linux and WSL2
<u>Sandbox runtime</u>	The whole Claude Code process, including file tools, MCP servers, and hooks	No	Low
<u>Dev container</u>	Full development environment	Yes	Medium
<u>Custom container</u>	Full development environment	Yes	Medium to high
<u>Virtual machine</u>	Full operating system	No	High

Approach	What is isolated	Requires Docker	Setup effort
<u>Claude Code on the web</u>	Full operating system, hosted by Anthropic	No	None; requires a Claude subscription and GitHub

The sandboxed Bash tool is built into Claude Code and restricts only Bash commands. Built-in file tools, MCP servers, and hooks still run directly on your host. Every other approach in the table puts the whole Claude Code process inside the isolation boundary, so file tools, MCP servers, and hooks are restricted too.



Sandbox isolation reduces the impact of a breach, but it does not eliminate risk. Any approach that allows network egress can still leak data the agent can read, and any approach that mounts your project directory writable can still modify that code. Review the security limitations before relying on a sandbox as a hard control.

Isolation also does not change what is sent to the model. Your prompts and the files Claude reads are transmitted to the Anthropic API or your configured provider with or without a sandbox. See Data usage for what Claude Code sends and how to reduce it.

Choose an approach

Match your goal to a row below, then read the detail section that follows.

You want to	Start with
Reduce permission prompts during everyday work on your own machine	The <u>sandboxed Bash tool</u> , enabled with <code>/sandbox</code>
Let Claude work unattended with <code>--dangerously-skip-permissions</code> or auto mode	The preconfigured <u>dev container</u> , any container or VM, or the <u>sandbox runtime</u>
Isolate MCP servers and hooks as well as Bash, without Docker	The sandbox runtime
Work on an untrusted repository	A dedicated virtual machine, or <u>Claude Code on the web</u> if you have a Claude subscription and a connected GitHub account
Standardize a sandboxed environment across a team	The preconfigured <u>dev container</u> , copied into your repository
Use Claude Code from a device with no local setup	<u>Claude Code on the web</u> , which requires a Claude subscription and a connected GitHub account
Require isolation for every developer in your organization	<u>Enforce isolation across an organization</u>
Work on a native Windows host	A container or VM, or run the Bash sandbox inside WSL2

How isolation relates to permission modes

Permission modes decide whether a tool call runs and whether you are prompted first. Isolation restricts what a command can access once it runs. The two work together: when a permission mode lets actions run without asking you, an isolation boundary limits what those actions can reach.

`--dangerously-skip-permissions` removes per-action review other than explicit **ask rules**, so an isolation boundary is the only thing limiting what Claude can do. Always run it inside a container, a VM, or the **sandbox runtime**, so that file tools, MCP servers, and hooks are also inside the boundary.

Auto mode replaces the prompt with a classifier that reviews actions and blocks ones that escalate beyond the request, target unrecognized infrastructure, or appear driven by hostile content Claude read. The classifier is a per-action control, not an isolation boundary, so an isolation boundary still adds defense in depth for unattended runs, and is not required the way it is for `--dangerously-skip-permissions`.

The **sandboxed Bash tool** on its own constrains only Bash, so it is not sufficient for fully unattended runs in either mode. You can layer approaches: running the sandboxed Bash tool inside a container or VM gives you OS-level command restrictions on top of the outer environment boundary. For how the Bash sandbox itself interacts with permission rules and modes, see **How sandboxing relates to permissions and permission modes**.

Sandboxed Bash tool

⚠ This option does not support native Windows. On Windows hosts, use WSL2 or one of the container or VM approaches below.

The sandboxed Bash tool is built into Claude Code. It uses operating system primitives to restrict the filesystem and network access of every Bash command Claude runs: Seatbelt, the built-in macOS sandbox, and **bubblewrap** on Linux and WSL2. By default it allows writes to the working directory and prompts the first time a command needs a new network domain.

Enable it with the `/sandbox` command. The **Sandboxing** guide covers the approval modes, the default boundary, and how to widen or narrow it.

The per-command sandbox does not cover everything that runs in a session:

- Other **built-in tools** such as Read, Edit, and WebFetch run inside the Claude Code process and do not spawn arbitrary code. **Permission rules** for path or domain gate them instead.
- **MCP** servers and hooks are separate processes that run unconstrained on the host.

To put built-in tools, MCP servers, and hooks all behind one OS boundary, run the whole Claude Code process inside the sandbox runtime, the dev container, or a custom container.

Sandbox runtime

The `@anthropic-ai/sandbox-runtime` package wraps an entire process in the same Seatbelt or bubblewrap isolation that the built-in Bash sandbox uses. Running Claude Code through it constrains every tool, hook, and MCP server in the session, not only Bash. The runtime is a beta research preview, and its configuration format may change as the package evolves.

The runtime denies all write and network access by default, so configure it before launching Claude Code through it. In `~/.srt-settings.json`, or a file you pass with `--settings`, allow write access to at least your project directory and Claude Code's configuration paths `~/.claude` and `~/.claude.json`. Allow the network domains your session needs, including `api.anthropic.com` or your configured provider's endpoint. See the package README for the full configuration schema.

Once the settings file is in place, launch Claude Code with `npx` and pass `claude` as the command to wrap:

```
npx @anthropic-ai/sandbox-runtime claude
```

Claude Code starts inside the sandbox with the filesystem and network boundaries you configured. The same command works for sandboxing standalone MCP servers or other helper processes.

Dev containers

A dev container runs Claude Code inside a Docker container that VS Code or a compatible editor manages, with your project mounted in. You can define your own with a `.devcontainer/` directory in your repository.

The claude-code repository publishes an example dev container with a default-deny iptables firewall as a starting point. Copy it into your repository and adjust the firewall allowlist, base image, and pinned Claude Code version to fit your environment. Because the firewall blocks unapproved egress, a configuration like this supports running Claude Code with `--dangerously-skip-permissions` for unattended work.

Custom container

You can run Claude Code in any Docker or OCI container image with your own network policies, mounted volumes, and seccomp profiles. This is the most common path for organizations with existing container infrastructure or CI runners.

Several managed sandbox and remote execution services can host the container for you. The same checklist applies as for any container you operate: review what is mounted writable, what credentials and tokens are reachable inside it, and what the network egress policy allows.

You can layer the built-in Bash sandbox inside the container for per-command restrictions.

Unprivileged containers need the nested-sandbox setting described in [**Sandboxing troubleshooting**](#).

Virtual machine

A dedicated virtual machine provides the strongest separation, with its own kernel and, in cloud or microVM deployments, its own virtualized hardware. Options include cloud instances, local hypervisors, and microVMs such as Firecracker.

Use this approach when you are evaluating untrusted code, when your security policy requires kernel-level separation between the agent and the host, or when no host-level approach meets your compliance requirements. Docker Desktop's [**sandboxes feature**](#) provides a microVM with its own Docker daemon and workspace sync, which can run Claude Code on hosts that already have Docker Desktop.

Claude Code on the web

[**Claude Code on the web**](#) runs each session in an isolated, Anthropic-managed virtual machine. A network proxy enforces a default allowlist, and a separate proxy holds your GitHub token outside the sandbox while issuing scoped credentials for repository access inside it.

Use this approach when you want full VM isolation without provisioning infrastructure yourself, or when you are delegating tasks from a device that does not have a local development environment. It requires a Claude subscription and a connected GitHub account, and sessions clone your repository from GitHub. See [**Claude Code on the web**](#) for plan availability and GitHub authentication options.

Enforce isolation across an organization

Individual developers can opt into any approach above. What an organization can enforce, and with which tools, depends on the approach:

- **Built-in Bash sandbox:** the only approach Claude Code enforces itself. Deliver the `sandbox` settings keys through managed settings, either as a file managed by your MDM or through server-managed settings on Claude.ai. See Enforce sandboxing with managed settings for the keys to deploy and how to keep developers from widening the policy.
- **Dev containers:** commit the example dev container to your repositories to standardize the environment across a team. This is a convention rather than an enforcement boundary, because Claude Code does not require a container. If developers should not be able to run Claude Code outside it, enforce that with your organization's device management or software allowlisting tools.
- **Custom containers and VMs:** distribute Claude Code through the approved image and use your organization's device management or software allowlisting tools to prevent installation outside it.

See also

These pages cover configuration and policy details for the approaches above.

- Sandboxing: configure the built-in sandboxed Bash tool
- Dev container: the preconfigured Docker development container
- Security: the full Claude Code security model
- Secure deployment: isolation guidance for Agent SDK applications
- Settings: all sandbox configuration keys, including managed settings delivery